

Projekt- und Architekturdokumentation

**Ernährungsplaner-Applikation**  
**„FitFuel Mobile“**

Zur Veranstaltung

’Bachelor of Science’ B.Sc.

im Studiengang ’Mobile Anwendungen’

vorgelegt dem Fachbereich

Informatik und Wirtschaftsinformatik

der

Provadis School of International Management and Technology

von

Lars Bolender, Fabian Gemming, Benedict Glatz

E053, E063, E151

BIN24

[lars.bolender@stud-provadis-hochschule.de](mailto:lars.bolender@stud-provadis-hochschule.de), [fabian.gemming@stud-provadis-hochschule.de](mailto:fabian.gemming@stud-provadis-hochschule.de),

[benedict.glatz@stud-provadis-hochschule.de](mailto:benedict.glatz@stud-provadis-hochschule.de)

Gutachter: Prof. Dr. Grebner

Ende der Bearbeitungsfrist: 23.03.2026

## Inhaltsverzeichnis

|                                                                     |            |
|---------------------------------------------------------------------|------------|
| <b>Tabellenverzeichnis .....</b>                                    | <b>iii</b> |
| <b>Abkürzungsverzeichnis .....</b>                                  | <b>iv</b>  |
| <b>1 Anforderungen und Ziele .....</b>                              | <b>1</b>   |
| 1.1 Themensteckbrief: Nutzer, Prozess, Pain und Kontext .....       | 1          |
| 1.2 Die Lösung .....                                                | 1          |
| 1.3 Details zum Softwareprojekt .....                               | 2          |
| 1.4 Entwicklungsfokus .....                                         | 2          |
| <b>2 Technik Stack .....</b>                                        | <b>3</b>   |
| 2.1 Rahmenbedingungen .....                                         | 3          |
| 2.2 Architektur-Entscheidungen .....                                | 3          |
| <b>3 Frontend: Struktur / Bausteine .....</b>                       | <b>3</b>   |
| 3.1 UI-Komponenten und technischer Aufbau .....                     | 4          |
| 3.1.1 Übersicht der Screens und Komponenten .....                   | 4          |
| 3.1.2 Styling-Konzept .....                                         | 5          |
| 3.1.3 Komponenten-Interaktion .....                                 | 5          |
| 3.2 Detailbetrachtung der Kernkomponente <i>DietScreen.js</i> ..... | 6          |
| 3.2.1 Lokale Hilfskonstrukte .....                                  | 7          |
| 3.3 State Management .....                                          | 7          |
| 3.3.1 Globaler State des Themings .....                             | 8          |
| 3.4 Routing und Navigation .....                                    | 8          |
| 3.4.1 Logik des bedingten Startbildschirms .....                    | 8          |
| 3.4.2 Individualisierte Bottom-Nav .....                            | 9          |
| 3.5 Persistenz .....                                                | 9          |
| 3.5.1 Relationales Datenbankschema (SQLite) .....                   | 9          |
| 3.5.2 Abstraktion und Migrationsstrategie .....                     | 10         |
| 3.6 Konfiguration .....                                             | 10         |
| 3.6.1 Metro Bundler Konfiguration .....                             | 10         |
| 3.7 Implementierung der Fachlogik .....                             | 11         |
| 3.7.1 Überblick .....                                               | 11         |
| 3.7.2 Grundumsatz: Harris-Benedict-Formel .....                     | 11         |
| 3.7.3 Gesamtenergiebedarf .....                                     | 11         |
| 3.7.4 Phasen-Logik und Kalorienziel .....                           | 12         |
| 3.7.5 Makronährstoff-Verteilung .....                               | 12         |
| <b>4 Tooling .....</b>                                              | <b>13</b>  |
| 4.1 Scripts in package.json .....                                   | 13         |

|           |                                                          |           |
|-----------|----------------------------------------------------------|-----------|
| 4.2       | Package Management .....                                 | 13        |
| 4.3       | Linters .....                                            | 14        |
| 4.4       | Formatter .....                                          | 14        |
| 4.5       | JSDoc .....                                              | 14        |
| 4.6       | TypeScript .....                                         | 14        |
| 4.7       | Branch-Struktur .....                                    | 14        |
| 4.8       | Builds & Deployment .....                                | 15        |
| <b>5</b>  | <b>Qualität .....</b>                                    | <b>16</b> |
| 5.1       | Unit Tests und Coverage .....                            | 16        |
| 5.2       | E2E Tests .....                                          | 17        |
| 5.3       | CI/CD: GitHub Actions Pipeline .....                     | 17        |
| 5.4       | Statische Codeanalyse und Dokumentation .....            | 17        |
| <b>6</b>  | <b>Quellcode Übersicht .....</b>                         | <b>18</b> |
| <b>7</b>  | <b>Detaillierte Aspekte .....</b>                        | <b>18</b> |
| 7.1       | Offline-First .....                                      | 18        |
| 7.2       | Design- / UI- / UX-Entscheidungen: Hauptnavigation ..... | 19        |
| <b>8</b>  | <b>Projektbericht .....</b>                              | <b>20</b> |
| 8.1       | Kapazitätsplan .....                                     | 20        |
| 8.2       | Herausforderungen .....                                  | 21        |
| 8.3       | Lessons Learned .....                                    | 21        |
| <b>9</b>  | <b>Anhang .....</b>                                      | <b>23</b> |
| 9.1       | Tabellen .....                                           | 23        |
| 9.2       | Abbildungen .....                                        | 29        |
| 9.3       | Quellcode-Ausschnitte .....                              | 30        |
| <b>10</b> | <b>KI-Verzeichnis .....</b>                              | <b>38</b> |
| 10.1      | Konzeption & Architektur-Entscheidungen .....            | 38        |
| 10.2      | Code-Generierung & Implementierung von Fachlogik .....   | 39        |
| 10.3      | UI/UX-Entwicklung & Komponenten-Styling .....            | 40        |
| 10.4      | Debugging & Problembehebung .....                        | 41        |
| 10.5      | Code-Qualität, Refactoring & Testing .....               | 42        |
| 10.6      | Textredaktion & Dokumentationsstruktur .....             | 43        |

**Tabellenverzeichnis**

|                                                             |    |
|-------------------------------------------------------------|----|
| Tabelle 1: Screen-Übersicht.....                            | 4  |
| Tabelle 2: PAL-Wert nach Aktivitätsstufe .....              | 12 |
| Tabelle 3: Standardabweichung pro Phase .....               | 12 |
| Tabelle 4: Überblick verwendeter npm-Skripte.....           | 13 |
| Tabelle 5: Testkategorien.....                              | 16 |
| Tabelle 6: Coverage-Statistik nach Verzeichnis .....        | 16 |
| Tabelle 7: Flow-Dateien aus Maestro .....                   | 17 |
| Tabelle 8: Codeverteilung nach Architektur-Schicht .....    | 18 |
| Tabelle 9: Verzeichnisstruktur und Schichtenzuordnung ..... | 23 |
| Tabelle 10: Tech Canvas.....                                | 25 |
| Tabelle 11: Übersicht der Komponenten .....                 | 26 |
| Tabelle 12: Tabellenübersicht der Datenbank .....           | 27 |
| Tabelle 13: Zeitplan (Soll/Ist).....                        | 28 |

## Abkürzungsverzeichnis

|             |                                                |
|-------------|------------------------------------------------|
| API.....    | Application Programming Interface              |
| BMR.....    | Basal Metabolic Rate                           |
| CI/CD.....  | Continuous Integration & Continuous Deployment |
| CRUD .....  | Create, Read, Update, Delete                   |
| EAS.....    | Expo Application Services                      |
| E2E.....    | End to End                                     |
| iOS .....   | iPhone Operating System                        |
| MBaaS ..... | Mobile Backend as a Service                    |
| MVP.....    | Minimum Viable Product                         |
| PAL.....    | Physical Activity Level                        |
| Props ..... | Properties                                     |
| QR.....     | Quick Response                                 |
| src.....    | Source                                         |
| SVG .....   | Scalable Vector Graphics                       |
| TDEE .....  | Total Daily Energy Expenditure                 |
| UI.....     | User Interface                                 |
| UX.....     | User Experience                                |

# 1 Anforderungen und Ziele

Für eine zielgerichtete Entwicklung der Applikation „**FitFuel Mobile**“ werden im folgenden Kapitel die Kernbedürfnisse der Zielgruppen sowie die daraus resultierenden Anforderungen definiert.

## 1.1 Themensteckbrief: Nutzer, Prozess, Pain und Kontext

FitFuel Mobile richtet sich primär an die Zielgruppe der jungen Erwachsenen im Alter von 25-40 Jahren, die als ambitionierte Hobby- oder Profisportler aktiv sind.

Der Kernprozess, den die App abbildet, umfasst den gesamten Workflow einer strukturierten **Fitness-Ernährung**. Dieser Prozess adressiert den zentralen **Nutzer-Pain**: Bestehende Apps trennen oft das Tracking von der Planung und der realen Vorratshaltung. FitFuel Mobile schließt diese Lücke durch einen geschlossenen Kreislauf, der sicherstellt, dass sportliche Ziele erreicht werden.

Der **Nutzungskontext** erstreckt sich vom Dashboard-Check am Morgen über die Planung der Woche bis hin zur aktiven Nutzung als Einkaufs- und Bestandsliste.

## 1.2 Die Lösung

FitFuel Mobile ist ein konfigurierbares Planungstool, das dem Nutzer die individuelle Kontrolle über seine Ernährungsplanung überlässt, anstatt auf vorgefertigte oder zufällig generierte Rezepte zurückzugreifen.

Das **Minimum Viable Product** (MVP) umfasst folgende Kernfunktionen:

1. **Cross-Platform-Entwicklung**: Implementierung für das iPhone Operating System (iOS), Android und Web.
2. **Bedarfsermittlung**: Kalkulation des Energie- und Makronährstoffbedarfs.
3. **Planung**: Erstellung einer eigenen Zutatendatenbank und darauf basierender Gerichte sowie die Organisation in einem tages- und wochenbasierten Mahlzeitenplaner.
4. **Logistik**: Vollautomatisierte Generierung von Einkaufslisten unter Berücksichtigung vorhandener Bestände (Stock-Management).

Zukünftige Erweiterungen nach Abschluss des MVPs umfassen:

- Einen QR-Code-Scanner zur automatischen Erfassung von Zutaten-Nährwerten.
- Einen Algorithmus zur dynamischen Kalorienanpassung basierend auf dem Gewichtsverlauf.

### 1.3 Details zum Softwareprojekt

FitFuel Mobile ist als plattformübergreifende Mobile-Applikation konzipiert, die auf eine konsistente **Nutzererfahrung** ausgelegt ist. Hinsichtlich des Vorgehensmodells wurde der agile Ansatz des Rapid Application Development (RAD) gewählt. Das schnelle **Prototyping** und iterative Arbeiten stehen im Fokus, um eine testbare Basisversion der Applikation zu erschaffen und flexibel auf Anforderungsänderungen reagieren zu können.

Zur strukturierten Umsetzung dieses Vorgehens wurde ein agiles Projektboard in **GitHub** aufgesetzt. Diese Planungsbasis ermöglichte es, die Implementierung der einzelnen Prototypen gezielt zu steuern, den **Projektfortschritt** iterativ anzupassen und zukünftige Feature-Erweiterungen realistisch in den Gesamtzeitplan einzugliedern.

Ein zentraler Baustein der Architektur ist der konsequente **Local-First-Ansatz**, der eine vollständige **Offline-Verfügbarkeit** sicherstellt (Die detaillierten konzeptionellen Beweggründe hierfür werden in Kapitel 7.1 Offline-First erläutert, die technische Realisierung der Datenpersistenz in Kapitel 3.5 Persistenz).

Zudem sorgt ein zentrales **State- und Theme-Management** dafür, dass die Applikationsdaten über alle Ansichten hinweg synchron bleiben und optische Anpassungen wie der Dark Mode global angewendet werden.

### 1.4 Entwicklungsfokus

Der primäre Entwicklungsfokus von FitFuel Mobile liegt auf der Reduzierung des manuellen Planungs- und Organisationsaufwands. Dies wird durch die Automatisierung von Kernprozessen an der Schnittstelle zwischen digitaler Ernährungsplanung und dem Einkauf realisiert.

Ein weiteres Kriterium ist die Flexibilität, nutzerspezifische Ernährungsstrategien abzubilden. Anstelle der Vorgabe statischer Pläne bietet das System Konfigurationsmöglichkeiten, um individuelle Alltagsstrukturen und Trainingsphasen abzubilden.

Zudem liegt ein Schwerpunkt auf der **User Experience (UX)** und der Fortschrittsvisualisierung durch Liniendiagramme für den Gewichtsverlauf sowie Fortschrittsringe für die tagesaktuelle Kalorien- und Makronährstoffabdeckung.

Die Architektur der Bedienoberfläche zielt darauf ab, die Anzahl der notwendigen Interaktionen bei der Erstellung von Wochenplänen und der Nutzung der Einkaufsliste zu minimieren. Dafür werden Vorlagen (**Presets**) implementiert, mit denen vollständige Mahlzeitpläne gespeichert und geladen werden können, anstatt jeden Tag manuell neu zu befüllen.

## 2 Technik Stack

Die technologische Basis von FitFuel Mobile wurde im Einklang mit dem Entwicklungsfokus (Siehe 1.4 Entwicklungsfokus) gewählt. Tabelle 10: Tech Canvas gibt einen detaillierten Überblick über die eingesetzten Frameworks, Bibliotheken und Versionsstände.

### 2.1 Rahmenbedingungen

FitFuel Mobile ist als plattformübergreifende Anwendung für iOS, Android und das Web konzipiert. Um dies effizient umzusetzen, fiel die Entscheidung auf die Verwendung des UI-Frameworks **React Native** in Kombination mit dem **Expo-Framework**. Dieser technologische Ansatz ermöglicht es, aus einer einzigen, gemeinsamen Codebasis heraus alle drei Zielplattformen zu bedienen. Als Programmiersprache kommt im Projekt **JavaScript** zum Einsatz.

### 2.2 Architektur-Entscheidungen

Aufgrund der Auslegung auf Offline-Verfügbarkeit existiert weder ein klassisches Backend noch eine Anbindung an ein Mobile Backend as a Service (MBaaS) oder externe Application Programming Interfaces (APIs).

Die nutzergenerierten Daten (z.B. Profil, Gewichtsverlauf, Zutatendatenbank etc.) werden ausschließlich lokal auf dem Endgerät gespeichert.

Die detaillierten technischen Umsetzungen der Architektur, insbesondere die plattformspezifische Datenpersistenz (SQLite und AsyncStorage), sind als Teil der Frontend-Architektur konzipiert und werden im folgenden Kapitel (Siehe 3 Frontend: Struktur / Bausteine) behandelt.

## 3 Frontend: Struktur / Bausteine

Der gesamte Quellcode der Anwendung ist unterhalb eines zentralen Source-Verzeichnisses (*src*) nach funktionalen Aspekten in sechs Unterordner aufgeteilt, wobei der Einstiegspunkt von *App.js* im Projektstamm gebildet wird (Siehe Tabelle 9: Verzeichnisstruktur und Schichtenzuordnung).

Diese Aufteilung spiegelt eine klare Drei-Schichten-Architektur wider.

- UI-Schicht (Screens, Components, Navigation, Theme)
- Fachlogik-Schicht (Calculations, Storage)
- Datenbank-Schicht (Database)



Die **UI-Schicht** ist zuständig für die Darstellung und Nutzerinteraktion. Screens koordinieren den Datenfluss und Zustand, wohingegen Components reine Darstellungseinheiten ohne Geschäftslogik abbilden.

Die **Fachlogik-Schicht** kapselt Berechnungen und den gesamten Datenbankzugriff. Screens greifen niemals direkt auf die Datenbank zu.

Die **Datenbank-Schicht** beinhaltet eine plattformspezifische Datenbank-Implementierung sowie statische Ausgangsdaten.

Durch diese Trennung lassen sich einzelne Schichten unabhängig voneinander austauschen, ohne Änderungen an den Screens zu erfordern (z.B. Theming).

### 3.1 UI-Komponenten und technischer Aufbau

Die Benutzeroberfläche der Anwendung basiert durchgehend auf funktionalen React-Komponenten und strukturiert sich entlang eines Navigationsgraphen in acht zentrale Screens.

#### 3.1.1 Übersicht der Screens und Komponenten

Jeder der acht Screens entspricht einer Route im Navigationsgraphen und übernimmt eine dezidierte Aufgabe innerhalb der Applikation.

| Screen                    | Aufgabe                                                                         |
|---------------------------|---------------------------------------------------------------------------------|
| <b>WelcomeScreen</b>      | Einstieg und App-Vorstellung beim Erststart                                     |
| <b>ProfileSetupScreen</b> | Erfassung von Nutzerstammdaten und Aktivitätsprofil                             |
| <b>DashboardScreen</b>    | Tagesübersicht mit Kalorienring, Makros und Gewichtsverlauf                     |
| <b>DietScreen</b>         | Wochen- und Tagesplanung der Mahlzeiten                                         |
| <b>CartScreen</b>         | Automatisch generierte Einkaufsliste aus dem Ernährungsplan                     |
| <b>DishesScreen</b>       | Create-, Read-, Update-, Delete- (CRUD)-Verwaltung benutzerdefinierter Gerichte |
| <b>IngredientsScreen</b>  | Verwaltung von Zutaten mit Lagerbestandsverfolgung                              |
| <b>SettingsScreen</b>     | App-Konfiguration (Thema, Datenverwaltung)                                      |

Tabelle 1: Screen-Übersicht

In diesen werden weitere wiederverwendbare Komponenten aufgerufen (Siehe Tabelle 11: Übersicht der Komponenten). Diese funktionalen Komponenten setzen den Hook-Mechanismus für das State Management und Side Effects ein. Die genaue Strategie wird in Kapitel 3.3 State Management beschrieben.

### 3.1.2 Styling-Konzept

Alle Stilangaben werden über die **React-Native-API** „StyleSheet.create()“ als geschlossene Objekte am Ende jeder Komponentendatei definiert. Die Styles werden einmalig beim Laden der Datei validiert und in eine interne Darstellung überführt, was zur Laufzeit schneller ist als inline-deklarierte Objektliterale.

Die Anwendung bindet sämtliche Icons als ressourcenschonende **Inline-SVG-Komponenten** direkt in den entsprechenden Dateien ein. Für komplexere Grafiken wie dem Kalorien-Fortschrittsring wird die SVG-Geometrie (z.B. Radius und Umfang) programmatisch ohne externe Bibliotheken berechnet und anschließend dargestellt. Die Liniendiagramme für den Gewichtsverlauf sowie die Fortschrittsringe werden nativ über die Bibliothek „**react-native-svg**“ gezeichnet (Vgl. Quellcode-Ausschnitt 1: Implementierung des Kalorienrings).

### 3.1.3 Komponenten-Interaktion

Die 15 Komponenten lassen sich in vier technische Gruppen einteilen, deren Funktionsweise die Interaktion bestimmen.

#### 3.1.3.1 Präsentationskomponenten (Stateless & Unidirectional)

Diese Komponenten halten keinen eigenen Zustand und führen keine Nebeneffekte (Side Effects) aus. Die Interaktion beschränkt sich auf den Empfang von Daten via Props und die Weitergabe von Benutzerinteraktionen über Callbacks an übergeordnete Schichten.

- **Rendering-Komponenten:** NutrientBar, MacroCard, CompactCalorieMacros, Button und InputField transformieren Props (Werte, Labels, Konfigurationen) in UI-Elemente.
- **Berechnungskomponenten:** Der CalorieRing fungiert als reine Berechnungskomponente, die aus Props die SVG-Geometrie ableitet.
- **Interaktive Präsentationskomponenten:** MealCard und ConfirmModal stellen Informationen dar und delegieren Benutzeraktionen primär über Callbacks (z.B. onPress) an die Parent-Komponente.

#### 3.1.3.2 UI-Zustandskomponenten (Lokaler State)

Diese Komponenten verwalten ihre eigenen, internen Daten.

- **PhaseIndicator:** Verwendet einen internen State zur Steuerung der Modal-Sichtbarkeit und zur temporären Zwischenauswahl von Datumsauswahlen, bevor diese bestätigt werden.

- **WeightInputModal:** Kapselt den Formulardaten-State (Gewicht, Datumsmodus). Die Kerninteraktion besteht in der Transformation und Validierung.
- **BottomNav:** Verwaltet die Tab-Reihenfolge im lokalen State und implementiert Touch-Interaktionen für Drag-and-Drop-Operationen.

### 3.1.3.3 Datenintegrierende Komponenten (Side Effects & Persistenz)

Sie interagieren autonom mit externen APIs, der Datenbank oder dem lokalen Speicher und kapseln Geschäftslogik.

- **WeightHistory:** Kommuniziert direkt mit dem Daten-Layer und verwaltet den State für Ansichtsmodi (Monat/Jahr) und den zugrundeliegenden Datensatz für das SVG-Rendering eigenständig.
- **BottomNav (Storage-Interaktion):** Liest beim Mount-Prozess asynchron die persistierte Tab-Reihenfolge über die AsyncStorage-API aus.
- **AddDishModal & AddIngredientModal:** Diese führen asynchrone Datenbankabfragen aus, um beim Öffnen verfügbare Zutaten zu laden. Im Editiermodus reichern sie bestehende Zutaten mit Nährwertdaten an. Die Komponente aggregiert Kalorien- und Makrowerte autonom.

### 3.1.3.4 Screen-Komponenten

Screens nehmen innerhalb der Komponentenhierarchie eine besondere Rolle ein. Sie koordinieren mehrere Komponenten, indem sie die Daten aus der Persistenzschicht laden, den lokalen Zustand verwalten und die daraus abgeleiteten Werte als Props an untergeordnete Komponenten weitergeben. Gleichzeitig empfangen sie Nutzerinteraktionen über Callbacks und stoßen daraufhin Schreiboperationen in der Datenbank an.

## 3.2 Detailbetrachtung der Kernkomponente *DietScreen.js*

Als exemplarische Analyse dient der Diet-Screen, der mit über 1300 Zeilen die komplexeste Ansicht der Anwendung darstellt. Er vereint Wochennavigation, Tages- und Wochenansicht, Mahlzeitenplanung und Zielwertanzeige in einem einzigen Screen.

Die UI-Elemente des Diet-Screens folgen zur Laufzeit einer mehrstufigen Verschachtelung (Siehe Abbildung 1: Diet-Screen Komponenten-Hierarchie).

Ausgehend vom **App-Navigator** teilt sich der Screen in mehrere logische und funktionale Blöcke auf. Den oberen Bereich der Ansicht bilden navigations- und kontextbezogene Komponenten wie der Header, die Wochennavigation sowie die DayTabs zur Auswahl des jeweiligen Wochentags.

Darunter schließt sich die **Zielwertanzeige** an, die sich in eine Kalorien-Zeile mit visuellem Farbindikator und eine Makro-Zeile zur Darstellung der Nährwertverteilung (Proteine, Kohlenhydrate, Fette) untergliedert.

Den zentralen Hauptbereich der Komponente bildet eine Scroll-View. Diese dient als Container für die vertikale Anordnung der Mahlzeiten-Slots (Breakfast, Lunch, Dinner sowie flexibel erweiterbare DynamicSlots).

Jeder dieser Slots kapselt wiederum die individuellen **Meal-Cards** der eingetragenen Gerichte (Siehe Quellcode-Ausschnitt 2: MealCards für den DietScreen). Abgeschlossen wird die Struktur auf der obersten Ebene des Screens durch feste Navigationselemente wie die Bottom-Nav sowie die beiden integrierten Overlay-Komponenten (*PresetMenuModal* und *DishPickerModal*), die bei Nutzerinteraktion über den Hauptinhalt gelegt werden.

### 3.2.1 Lokale Hilfskonstrukte

Bevor die eigentliche Diet-Screen-Funktion beginnt, definiert die Datei inline mehrere funktionale Hilfskomponenten (wie *FoodIcon*, *DragHandle*, *PlusIcon*, *CloseIcon*, *TrashIcon*) sowie ausgelagerte Hilfsfunktionen (z.B. *formatDate* und *getWeekDates*). Diese Bausteine werden bewusst lokal in der Datei gehalten, da sie ausschließlich im visuellen Kontext dieses Screens benötigt werden.

## 3.3 State Management

Der Zustand der Applikation wird über React-eigene Hooks (z.B. *useState*, *useEffect* etc.) sowie die **React Context API** verwaltet. Der gesamte Anwendungszustand ist ausschließlich lokal in den jeweiligen Screens und Komponenten abgelegt und die Datenbank gilt als **Single Source of Truth**.

Jeder Screen verwaltet seinen eigenen State unabhängig voneinander. Die Kategorien des lokalen States lassen sich dabei unterscheiden:

- **Datenzustand:** Geladene Entitäten aus der Datenbank, z. B. *mealPlan*, *dishes*, *profile*, *weightHistory*. Diese werden beim Fokussieren des Screens asynchron befüllt.

- **UI-Zustand:** Sichtbarkeit von Modalen (*showDishPicker*, *showPresetMenu*), der aktive Tab (*activeTab*), der aktuelle Ansichtsmodus (*viewMode*).
- **Formularzustand:** Eingabefelder und temporäre Werte, die erst bei Bestätigung persistiert werden, z. B. Gewichtseingaben oder Profelfelder.

Abgeleitete Werte (wie Tagessummen, Makroziele oder Farbindikatoren) werden nicht als eigener State gehalten, sondern zur Renderzeit synchron aus dem vorhandenen State berechnet. Dies vermeidet inkonsistente Zustände, bei denen State-Variablen aus dem Takt geraten könnten.

### 3.3.1 Globaler State des Themings

Die einzige Cross-Screen-Zustandsverwaltung ist das Theming. Es wurde mit der **React Context API** realisiert, da dies der von React vorgesehene Mechanismus für die Bereitstellung von Werten durch den gesamten Komponentenbaum ist.

Der ThemeProvider wird in *App.js* als äußerste Hülle um den AppNavigator gelegt, sodass alle Screens und Komponenten via *useTheme()* auf die Props zugreifen können (Siehe Quellcode-Ausschnitt 3: Theming). Die Farbpalette wechselt damit in der gesamten Anwendung synchron, ohne dass Props über mehrere Ebenen weitergereicht werden müssen. Die Präferenz wird in AsyncStorage gespeichert und beim nächsten App-Start wiederhergestellt.

## 3.4 Routing und Navigation

Die Architektur der Navigation wird zentral in der Datei *AppNavigator.js* durch einen nativen Stack-Navigator der Bibliothek *React Navigation* (Version 7) realisiert (Siehe Quellcode-Ausschnitt 4: App-Navigation). Alle Screens implementieren ihre eigenen Kopfzeilen als Teil der Komponente, um vollständige Kontrolle über das Layout zu behalten. Die Ausnahme bildet der ProfileSetup-Screen, der einen minimalen nativen Header mit Zurück-Button erhält.

### 3.4.1 Logik des bedingten Startbildschirms

Der App-Navigator verfügt über einen sogenannten **Onboarding-Guard** zur Steuerung des App-Starts. Über den Hook *useEffect* wird mittels der asynchronen Funktion *loadUserProfile()* initial geprüft, ob bereits ein Nutzerprofil mit hinterlegtem Namen existiert. Erst nach Abschluss dieser Verifikation wird der Navigations-Container gerendert, wobei das Prüfergebnis den *initialRouteName* determiniert (Siehe Quellcode-Ausschnitt 5: Initialisierung des AppNavigators).

Dieses Muster stellt sicher, dass ein Nutzer beim Erststart den Onboarding-Flow durchläuft und danach direkt auf dem Dashboard landet, ohne dass die Welcome- und ProfileSetup-Screens erneut durchlaufen werden müssen.

### 3.4.2 Individualisierte Bottom-Nav

Anstelle eines Tab-Navigators aus der React-Navigation-Bibliothek wurde eine eigene Bottom-Nav-Komponente entwickelt. Diese unterscheidet sich in einem wesentlichen Merkmal: Die Reihenfolge der fünf Tabs (Planner, Cart, Home, Dishes, Ingredients) ist per **Drag-and-Drop** durch den Nutzer anpassbar.

Die Navigation selbst wird durch die individualisierte Navigation nicht durch den Navigator gesteuert, sondern durch einen *handleTabPress*-Callback, den jeder Screen dem Bottom-Nav übergibt.

## 3.5 Persistenz

Um die plattformübergreifenden Anforderungen des Local-First-Ansatzes zu erfüllen, ist die Datenbankschicht in zwei spezifische Dateien unterteilt. Der **Metro-Bundler** von Expo wählt zur Build-Zeit automatisch die für die jeweilige Laufzeitumgebung passende Datei aus. Während für native Builds (iOS/Android) die Datei *database.native.js* priorisiert wird, greift im Web-Kontext automatisch die *database.web.js*.

Die zentrale *database.js* fungiert dabei lediglich als **Re-Export-Schnittstelle**. Dieser Ansatz ermöglicht es dem restlichen Anwendungscode, auf eine einheitliche API zuzugreifen, ohne dass fehleranfällige Laufzeit-Prüfungen erforderlich sind.

### 3.5.1 Relationales Datenbankschema (SQLite)

Auf den nativen Plattformen (iOS und Android) wird eine **Expo-SQLite-Datenbank** (fit-fuel.db) mit **WAL-Journaling** (Write-Ahead Logging) verwendet. Dabei wird ein Log erstellt, der zeitlich versetzt auf die Datenbank übertragen wird. Dies verhindert unter anderem, dass beim Eintragen mehrerer Zutaten in einem kleinen Zeitfenster die UI einfriert.

Das Schema umfasst **zwölf Tabellen**, die durch Fremdschlüsselbeziehungen und Cascading die **referenzielle Integrität** sicherstellen (Siehe Tabelle 12: Tabellenübersicht der Datenbank).

Für datumbasierte und häufig gefilterte Abfragen wurden sieben Indizes angelegt (Vgl. Quellcode-Ausschnitt 6: Indizes der Datenbank).

### 3.5.2 Abstraktion und Migrationsstrategie

Da SQLite im Browser nicht zur Verfügung steht, dient der Key-Value-Speicher AsyncStorage als Fallback. Für einen nahtlosen Übergang wurde eine Migrationsroutine implementiert, die bestehende AsyncStorage-Daten beim ersten Start in die SQLite-Datenbank überführt.

Die Datei *storage.js* bildet die einzige nach außen sichtbare Schnittstelle der Datenhaltung. Sie importiert alle Low-Level-Funktionen aus *database.js* und stellt sie unter vereinfachten Namen bereit (*saveUserProfile*, *loadAllDishes*, *getMealPlanForDate* etc.).

Beim ersten App-Start nach einem Update auf die SQLite-Version führt *initDatabase()* automatisch eine Einwegmigration von AsyncStorage nach SQLite durch. Die **Migrationsfunktion** prüft über einen Flag in AsyncStorage, ob die Migration bereits stattgefunden hat.

## 3.6 Konfiguration

Die Expo-Konfiguration in *app.json* definiert die zentralen Metadaten und Plattformparameter der Anwendung. Zwei Einträge sind für die Funktionalität der App besonders relevant:

- **"newArchEnabled": true** aktiviert die neue React-Native-Architektur, die direkten synchronen Zugriff zwischen JavaScript und nativen Modulen ermöglicht und damit die Grundlage für die expo-sqlite-Integration bildet.
- Der **"plugins"**-Eintrag registriert expo-sqlite als natives Expo-Plugin, das beim Build-Prozess die notwendigen nativen Konfigurationen (Für Apple Xcode und für Android Gradle) automatisch einbettet, ohne dass manuelle native Konfigurationsdateien angefasst werden müssen.

Die **Bildschirmausrichtung** ist auf „portrait“ festgelegt, da das UI-Layout der Screens nicht für die Landscape-Darstellung ausgelegt ist. Während ein solches kantenloses Vollbild-Design unter iOS bereits der systemseitige Standard ist, gibt *edgeToEdgeEnabled* auf Android der App explizit den gesamten Bildschirmbereich einschließlich der Systemleisten frei. In Verbindung mit der *SafeAreaView* sorgt dies plattformübergreifend in jedem Screen für ein konsistentes Layout ohne verdeckte Inhalte.

### 3.6.1 Metro Bundler Konfiguration

Als nativer JavaScript-Bundler für React Native ist **Metro** dafür verantwortlich, alle Module und Assets der Anwendung zu analysieren und für die jeweilige Zielpattform zusammenzufassen.

Die Datei *metro.config.js* erweitert die Expo-Standardkonfiguration um eine benutzerdefinierte *resolveRequest*-Funktion. Diese wurde eingeführt, um fehlerhafte Pfade zu erkennen und zur korrekten Verarbeitung explizit an den Standard-Resolver zurückzudelegieren (Vgl. Quellcode-Ausschnitt 8: *metro.config.js*).

Die plattformspezifische Dateiauflösung (*database.native.js* vs. *database.web.js*) ist kein benutzerdefiniertes Verhalten, sondern wird vom Standard-Metro-Resolver über die eingebaute Suffix-Auflösung behandelt.

## 3.7 Implementierung der Fachlogik

### 3.7.1 Überblick

Die gesamte ernährungswissenschaftliche Berechnungslogik ist in *calculations.js* isoliert. Die Datei enthält ausschließlich Funktionen ohne Side-Effects, State oder UI-Abhängigkeiten. Screens importieren die benötigten Funktionen und übergeben Nutzerdaten als Parameter.

### 3.7.2 Grundumsatz: Harris-Benedict-Formel

Der Grundumsatz (BMR, Basal Metabolic Rate) wird nach der Harris-Benedict-Formel berechnet, getrennt nach Geschlecht:

$$\text{BMR}_{\text{Mann}} = 66,473 + (13,752 \times m) + (5,003 \times h) - (6,755 \times a)$$

$$\text{BMR}_{\text{Frau}} = 655,096 + (9,563 \times m) + (1,850 \times h) - (4,676 \times a)$$

wobei  $m$  das Körpergewicht in kg,  $h$  die Körpergröße in cm und  $a$  das Alter in Jahren ist. Das Ergebnis wird auf ganze Kilokalorien gerundet. Fehlen Pflichtparameter, gibt die Funktion 0 zurück.

### 3.7.3 Gesamtenergiebedarf

Der Gesamtenergiebedarf (TDEE, Total Daily Energy Expenditure) ergibt sich aus der Multiplikation des BMR mit dem gewichteten Physical Activity Level (PAL-Faktor).

$$\text{TDEE} = \text{BMR} \times \text{PAL}_{\text{gesamt}}$$

Das Aktivitätsprofil des Nutzers besteht aus sechs Aktivitätsstufen, denen jeweils ein mittlerer PAL-Wert und wie in Tabelle 2: PAL-Wert nach Aktivitätsstufe zu sehen eine Stundenanzahl zugeordnet werden.



| Aktivitätsstufe        | PAL-Wert | Standardstunden (in h) |
|------------------------|----------|------------------------|
| Schlafen               | 0,95     | 8                      |
| Sitzen/Liegen          | 1,20     | 0                      |
| Sitzen                 | 1,45     | 9                      |
| Sitzen mit Gehpausen   | 1,65     | 3                      |
| Gehen/Stehen           | 1,85     | 2                      |
| Körperlich anstrengend | 2,20     | 2                      |

Tabelle 2: PAL-Wert nach Aktivitätsstufe

Der Tages-PAL ergibt sich als stundengewichteter Durchschnitt über alle sechs Stufen:

$$\text{PAL}_{\text{gesamt}} = \frac{\sum_i (\text{Stunden}_i \times \text{PAL}_i)}{24}$$

Die Summe aller Stunden muss exakt 24 ergeben; andernfalls wird 0 zurückgegeben, um unplausible Eingaben abzufangen.

### 3.7.4 Phasen-Logik und Kalorienziel

Aus dem TDEE wird das persönliche Kalorienziel abhängig von der gewählten Ernährungsphase berechnet:

| Aktivitätsstufe | Standardabweichung (in %) | Gültigkeitsbereich   |
|-----------------|---------------------------|----------------------|
| Bulk            | +10                       | 0 % bis +50 %        |
| Cut             | -15                       | -50 % bis 0 %        |
| Maintenance     | +3,5                      | fest, nicht änderbar |

Tabelle 3: Standardabweichung pro Phase

Der Prozentwert ist durch den Nutzer in den Einstellungen innerhalb der angegebenen Grenzen anpassbar (Siehe Gültigkeitsbereich in Tabelle 3: Standardabweichung pro Phase). Das Minimum des Kalorienziels ist auf 1.200 kcal begrenzt.

### 3.7.5 Makronährstoff-Verteilung

Aus dem berechneten Kalorienziel werden die Makronährstoffziele nach einem festen prozentualen Schlüssel abgeleitet:

$$\text{Protein} = \frac{\text{kcal} \times 0,30}{4}$$

$$\text{Kohlenhydrate} = \frac{\text{kcal} \times 0,45}{4}$$

$$\text{Fett} = \frac{\text{kcal} \times 0,25}{9}$$

Die Divisoren entsprechen den physiologischen Brennwerten der Makronährstoffe in kcal/g.

## 4 Tooling

Es wird ein **React-Native-Stack** auf Basis von Expo eingesetzt. Dieses Tooling bildet sowohl die lokale Entwicklungsumgebung als auch die Qualitätssicherung und zentralen Build-Prozesse ab. Die Konfiguration erfolgt primär über die Datei *package.json*, ergänzt durch projektspezifische Einstellungen in *app.json* sowie Regeln zur statischen Codeanalyse in *eslinttrc.json*.

### 4.1 Scripts in package.json

Die wichtigsten Entwicklungs- und Qualitätsprozesse sind über npm-Skripte standardisiert.

| Script                         | Nutzen                                                        |
|--------------------------------|---------------------------------------------------------------|
| <b>npm start</b>               | Startet den Expo-Entwicklungsserver                           |
| <b>npm run android</b>         | Startet die App auf einem Android-Gerät oder Emulator         |
| <b>npm run ios</b>             | Startet die App auf einem iOS-Simulator                       |
| <b>npm run web</b>             | Startet die Web-Variante über Expo                            |
| <b>npm test</b>                | Führt die Unit- und Component-Tests mit Jest aus              |
| <b>npm run test:coverage</b>   | Erstellt zusätzlich einen Coverage-Report                     |
| <b>npm run test:e2e</b>        | Führt End-to-End (E2E)-Tests mit Maestro aus                  |
| <b>npm run test:e2e:report</b> | Erstellt aus den E2E-Ergebnissen zusätzlich einen HTML-Report |
| <b>npm run lint</b>            | Prüft den Quellcode mit ESLint                                |
| <b>npm run lint:fix</b>        | Behebt automatisch einen Teil der ESLint-Verstöße             |

Tabelle 4: Überblick verwendeter npm-Skripte

### 4.2 Package Management

Als Package Manager wird npm verwendet, das die Installation, Versionierung sowie die Auflösung sämtlicher Projektabhängigkeiten übernimmt. Zu den zentralen produktiven Abhängigkeiten zählen insbesondere Expo als Entwicklungsplattform für React Native sowie React und React Native als grundlegende Basis der Anwendung.

Die Mehrsprachigkeit der Anwendung wird durch die Kombination aus *expo-localization*, *i18next* und *react-i18next* umgesetzt. Ergänzend kommen im Entwicklungsprozess Werkzeuge wie **ESLint** zur statischen Codeanalyse sowie **Jest**, *Jest-Expo* und *Testing Library* sowie *React Native* für **automatisierte Tests** zum Einsatz.

#### 4.3 Linter

Zur statischen Codeanalyse wird **ESLint** eingesetzt. Die Konfiguration befindet sich in *eslinttrc.json*.

Zusätzlich sind Plugins für *React*, *React Hooks* und *React Native* eingebunden. Damit werden typische Fehler früh erkannt, zum Beispiel ungenutzte Variablen, fehlende *const*-Verwendung und Regelverstöße bei *React Hooks*.

#### 4.4 Formatter

Ein dedizierter Code-Formatter wie **Prettier** ist im aktuellen Projektstand nicht integriert. Stattdessen werden grundlegende Formatierungsregeln über **ESLint** abgebildet.

#### 4.5 JSDoc

In der ESLint-Konfiguration ist *jsdoc/recommended* als Plugin eingebunden. Zusätzlich wird mit *jsdoc/require-jsdoc* verlangt, dass Funktionen und Methoden dokumentiert werden. Beschreibungen für Parameter und Rückgabewerte werden ebenfalls geprüft.

**JSDoc** wird vor allem in Bereichen eingesetzt, in denen Funktionen oder Module erklärungsbedürftig sind, zum Beispiel Utility-Funktionen in *src/utills/*, Datenmodule wie *src/data/food-Database.js* und *src/data/mealPlans.js*, Komponenten, Screens und Datenbankzugriffe in *database.native.js* und *database.web.js*.

#### 4.6 TypeScript

Auf eine strikte Typisierung durch **TypeScript** wird hierbei verzichtet, um die Geschwindigkeit beim Prototyping und somit den Fokus auf das MVP zu erhöhen.

#### 4.7 Branch-Struktur

Die Organisation des Quellcodes erfolgt über ein Branching-Modell mit klarer Trennung zwischen stabilen Zuständen (**main-Branch**) und in Entwicklung Befindlichen (**dev-Branch**).

Der **main-Branch** umfasst ausschließlich lauffähige und stabile Versionen der Anwendung.

Die eigentliche Entwicklung findet auf einem separaten **dev-Branch** statt. Dieser dient als Integrationszweig für neue Funktionalitäten und stellt den aktuellen Entwicklungsstand der Anwendung dar.

Für die Implementierung einzelner Features werden zusätzlich **Feature-Branches** verwendet, die jeweils vom dev-Branch abzweigen. Jeder dieser Branches fokussiert sich auf eine klar abgegrenzte Funktionalität (z. B. einzelne Screens oder Komponenten).

#### 4.8 Builds & Deployment

Dev Builds werden über den Befehl `expo run:android` (bzw. `expo run:ios`) als **lokaler Pre-build** kompiliert, Expo generiert dabei nativen Android/iOS-Code und baut die App direkt auf dem angeschlossenen Gerät oder Emulator. Alternativ wird `expo start` genutzt, um den Entwicklungsserver zu starten und die App über die **Expo Go App** auf dem Handy zu testen.

Ein fertiger Production Build (`.apk/.aab` für Android oder `.ipa` für iOS) wurde im Rahmen dieses Projekts nicht generiert. Für eine zukünftige Veröffentlichung im Google Play Store oder Apple App Store wird der **Expo Application Services (EAS Build)** eingesetzt, da EAS eine nahtlose Cloud-basierte Build-Pipeline bietet und sich direkt in das bestehende Expo-Projekt integrieren lässt, ohne eine lokale native Build-Umgebung einrichten zu müssen.

## 5 Qualität

Das Projekt verfolgt eine mehrstufige Teststrategie aus **Unit Tests**, **End-to-End-Tests**, einer **CI/CD-Pipeline** sowie **statischer Codeanalyse** mit verpflichtender **Dokumentation**.

### 5.1 Unit Tests und Coverage

Als Testframework dient **Jest** mit dem Preset *jest-expo*. Die Tests verteilen sich auf vier Kategorien und werden über *Jest* automatisch ausgewertet:

| Verzeichnis        | Inhalt                                                         | Testanzahl |
|--------------------|----------------------------------------------------------------|------------|
| <b>utils/</b>      | Berechnungslogik (BMR, TDEE, Makros, Alter, Phasen),<br>Logger | 39         |
| <b>data/</b>       | Lebensmitteldatenbank, Mahlzeitenplan-Hilfsfunktionen          | 13         |
| <b>components/</b> | Button, InputField, MacroCard, CalorieRing                     | 18         |
| <b>screens/</b>    | WelcomeScreen, DashboardScreen                                 | 11         |

Tabelle 5: Testkategorien

Die Coverage-Statistik spiegelt die bewusste Priorisierung der Geschäftslogik wider. Die Datenschicht erreicht 100 % über alle Metriken, während UI-Screens im MVP durch E2E-Tests abgedeckt werden.

| Verzeichnis        | Statement (in %) | Branches (in %) | Functions (in %) | Lines (in %) |
|--------------------|------------------|-----------------|------------------|--------------|
| <b>data/</b>       | 100              | 100             | 100              | 100          |
| <b>theme/</b>      | 67,64            | 37,5            | 44,44            | ~70          |
| <b>components/</b> | 20,48            | 18,35           | 16,25            | 21,15        |
| <b>screens/</b>    | 6,42             | 3,16            | 4,44             | 6,9          |
| <b>navigation/</b> | 0                | 0               | 0                | 0            |
| <b>Gesamt</b>      | 12,67            | 12,51           | 12,64            | 12,92        |

Tabelle 6: Coverage-Statistik nach Verzeichnis

## 5.2 E2E Tests

Für End-to-End-Tests kommt **Maestro** zum Einsatz, das Testszenarien deklarativ in **YAML** beschreibt. Die vier Flows im Verzeichnis `.maestro/` decken die zentralen Nutzerpfade ab:

| Flow-Datei                   | Zweck                                                       | Dauer (in Sek.) |
|------------------------------|-------------------------------------------------------------|-----------------|
| <i>01_welcome.yaml</i>       | Welcome-Screen-Navigation und Weiterleitung zum Onboarding  | 19              |
| <i>02_profile_setup.yaml</i> | Vollständiges Ausfüllen des Profilformulars mit Validierung | 50              |
| <i>03_dashboard.yaml</i>     | Dashboard-Elemente nach Login verifizieren                  | 34              |
| <i>04_navigation.yaml</i>    | Bottom-Navigation und Tab-Wechsel                           | 48              |

Tabelle 7: Flow-Dateien aus Maestro

Jeder Flow startet in einem bereinigten Zustand (u.a. zurückgesetzte Datenbank). Die Ausführung erfolgt über `npm run test:e2e`, der Report wird via **junit2html** unter `reports/e2e-report.html` erzeugt. Alle vier Flows werden auf einem Pixel 9 Pro XL Emulator durchlaufen.

## 5.3 CI/CD: GitHub Actions Pipeline

Eine GitHub Actions Pipeline (`.github/workflows/ci.yml`) wird bei jedem Push und Pull Request auf main ausgelöst und umfasst vier Jobs.

Zunächst führt der Job *lint* die statische Codeanalyse durch. Ist dieser erfolgreich, startet der Job *test* die 81 Unit- und Komponententests. Im Anschluss generiert der Job *coverage* den Report und lädt ihn als Pipeline-Artefakt hoch. Parallel zu dieser sequenziellen Kette läuft der Job *build-web*, der die Web-Kompilierbarkeit über Expo abprüft.

## 5.4 Statische Codeanalyse und Dokumentation

ESLint erzwingt über `eslinttrc.json` einheitliche Formatierungsregeln (vier Leerzeichen Einrückung, einfache Anführungszeichen, Semikolons, `const/let` statt `var`) sowie korrekte Hook-Verwendung.

Über ein ESLint-Plugin (`eslint-plugin-jsdoc`) stellen wir sicher, dass JSDoc vollständig umgesetzt wurde. Dies umfasst eine verpflichtende JSDoc-Dokumentation für alle exportierten Funktionen konfiguriert, einschließlich beschriebener Parameter und Rückgabewerte. Testdateien sind davon ausgenommen.

## 6 Quellcode Übersicht

Die Anwendung unterstützt nativ drei Plattformen (iOS, Android, Web) in den beiden Sprachen Englisch und Deutsch. Die lokale Persistenz erfolgt über zwölf SQLite-Tabellen mit sechs Indizes. Zur Qualitätssicherung dienen Unit/Komponententests sowie E2E-Flows.

Die Codebasis umfasst 37 JavaScript-Quelldateien mit 11.691 Zeilen (ohne Tests), ergänzt durch 10 Testdateien (659 Zeilen) und 4 Maestro-E2E-Flows (94 Zeilen YAML) – insgesamt 51 Dateien mit 12.444 Zeilen.

Der dominierende UI-Anteil besteht zu 66,1 % aus Komponenten und Screens. Er entspricht dem Charakter einer frontend-lastigen Anwendung ohne Backend. Die schlanke Fachlogik-Schicht belegt die konsequente Trennung von Berechnung und Darstellung:

| Verzeichnis        | Dateien | Zeilen | Anteil (in %) |
|--------------------|---------|--------|---------------|
| <b>components/</b> | 15      | 4.112  | 35,2          |
| <b>screens/</b>    | 8       | 3.616  | 30,9          |
| <b>utils/</b>      | 8       | 1.080  | 9,2           |
| <b>data/</b>       | 2       | 300    | 2,6           |
| <b>theme/</b>      | 4       | 192    | 1,7           |

---

Tabelle 8: Codeverteilung nach Architektur-Schicht

---

## 7 Detaillierte Aspekte

Nachdem in den vorangegangenen Abschnitten die technischen und architektonischen Grundlagen der Anwendung beleuchtet wurden, fokussiert sich dieses Kapitel auf die zugrundeliegenden konzeptionellen Entscheidungen. Im Folgenden wird dargelegt, aus welchen nutzerzentrierten Beweggründen die Schwerpunkte gezielt auf die Offline-First-Architektur sowie auf spezifische Design- und UX-Entscheidungen gelegt wurden.

### 7.1 Offline-First

Ein zentraler Aspekt der User Experience von FitFuel Mobile ist die Umsetzung des Offline-First- bzw. **Local-First**-Ansatzes. Da die Applikation typischerweise in Umgebungen mit potenziell schwankender Netzqualität genutzt wird, sind sämtliche Kernfunktionen, wie die Ernährungsplanung, Gewichtsdocumentation und die Verwaltung von Zutaten, vollständig ohne Internetverbindung nutzbar.

Der Verzicht auf eine externe Backend-Infrastruktur bietet architektonische und funktionale Vorteile: Die Anwendung weist kurze Ladezeiten auf und reagiert unmittelbar auf Nutzereingaben, da Wartezeiten durch Netzwerklatenzen entfallen.

Darüber hinaus unterstützt die lokale Datenhaltung den konzeptionellen Fokus auf hochgradig **personalisierte Ernährungsdaten**. Anstatt über eine Schnittstelle auf eine umfassende, aber oft unübersichtliche webbasierte Lebensmitteldatenbank zuzugreifen, pflegt der Nutzer einen eigenen, auf seine individuellen Präferenzen zugeschnittenen Zutatenbestand. Dies reduziert den Suchaufwand bei der täglichen Planung und verhindert eine Überladung der Benutzeroberfläche mit irrelevanten oder redundanten Datensätzen.

Zudem ist die lokale Datenhaltung ein bewusster Bestandteil des „**Privacy by Design**“-Prinzips. Der Verzicht auf eine Cloud-Anbindung und geräteübergreifende Synchronisation stellt sicher, dass die Datenhoheit beim Nutzer verbleibt. Nutzergenerierte Gesundheits-, Gewichts- und Ernährungsdaten werden ausschließlich lokal auf dem Endgerät gespeichert. Dieser Ansatz priorisiert den Datenschutz sowie die Unabhängigkeit der Anwendung von externer Infrastruktur.

## 7.2 Design- / UI- / UX-Entscheidungen: Hauptnavigation

Ein zentraler Aspekt der UX-Gestaltung ist die Konzeption der Hauptnavigation. Ziel war es, die wichtigsten Funktionsbereiche logisch gruppiert und im Nutzungskontext intuitiv verständlich anzuordnen. Dafür wurde eine Bottom-Navigation implementiert, die eine ergonomische **Einhandbedienung** im unteren Bildschirmbereich ermöglicht.

Die bewusste Begrenzung auf exakt fünf Hauptbuttons stellt sicher, dass alle zentralen Funktionen dauerhaft sichtbar bleiben.

Ursprünglich war in Betracht gezogen worden, die Vorlagenverwaltung (**Presets**) als sechsten Tab zu integrieren. Diese Idee wurde jedoch zugunsten der Ergonomie verworfen: Zu viele Touch-Ziele auf dem begrenzten horizontalen Raum eines Smartphone-Displays erhöhen die Fehleranfälligkeit bei der Eingabe (das sogenannte „Fat-Finger-Problem“) und mindern die visuelle Übersichtlichkeit.

Stattdessen wurde das Preset-Menü als kontextbezogenes Overlay direkt in den Planungs-Screen integriert, wo es sich nahtlos in den aktiven Workflow einfügt.

Die Default-Anordnung der Funktionsbereiche folgt einer nutzungsorientierten Struktur:



- **Zentrum (Dashboard):** Die mittlere Position dient als primärer Einstiegspunkt. Durch diese Platzierung erhält das Dashboard visuell die höchste Priorität und ermöglicht einen schnellen Zugriff auf den aktuellen Status.
- **Linke Seite (Planner und Cart):** Diese Seite bündelt die operativen Kernfunktionen. Da benötigte Zutaten im Einkaufswagen häufig direkt aus der Ernährungsplanung resultieren, reduziert diese räumliche Nähe Kontextwechsel.
- **Rechte Seite (Dishes und Ingredients):** Diese Bereiche dienen primär der Verwaltung und Datenpflege. Die Trennung in eine planungsorientierte linke und eine verwaltungsorientierte rechte Seite verbessert die Orientierung.

Die visuelle Gestaltung unterstützt diese Struktur: Jeder Tab kombiniert ein Icon mit einem Textlabel für eine optimale Erkennbarkeit. Eine farbliche Hervorhebung und erhöhte Schriftgewichtung des aktiven Tabs sorgen für eine eindeutige Zustandsrückmeldung.

Ergänzend wurde die Navigation personalisierbar gestaltet, sodass die Tab-Reihenfolge per **Drag-and-Drop** an individuelle Nutzungsschwerpunkte angepasst werden kann. Diese funktionale Flexibilität dient primär der Barrierefreiheit und der ergonomischen Einhandbedienung: Links- und Rechtshänder können die für sie wichtigsten Interaktionsflächen gleichermaßen in die optimal erreichbaren Bildschirmbereiche verschieben.

Eine weitere bewusste Designentscheidung war es, die **App-Einstellungen** aus der Hauptnavigation auszuschließen und im Dashboard zu platzieren. Dies verhindert eine Überladung der Navigation und priorisiert den schnellen Zugriff auf die Kernfunktionen.

## 8 Projektbericht

### 8.1 Kapazitätsplan

Im Rahmen der Projektplanung wurde ein strukturierter Zeitplan erstellt, der die einzelnen Entwicklungsphasen sowie die zugehörigen Arbeitspakete der Applikation „FitFuel Mobile“ definiert. Ziel war es, eine klare zeitliche Struktur für die Umsetzung der Kernfunktionalitäten des **MVP** zu schaffen und die verfügbaren Ressourcen innerhalb des Teams effizient zu nutzen.

Die Umsetzung erfolgte iterativ auf Basis des gewählten Rapid Application Development (RAD) Ansatzes. Einzelne Funktionsbereiche wurden zunächst als eigenständige Prototypen

implementiert und im weiteren Verlauf schrittweise erweitert sowie in die Gesamtanwendung integriert.

Die geplanten Soll-Zeiten der Arbeitspakete konnten weitgehend eingehalten werden. Abweichungen ergaben sich jedoch im Bereich der Projekt- und Architekturdokumentation. Für deren Erstellung wurde mehr Zeit benötigt als ursprünglich vorgesehen, wodurch der zeitliche Aufwand für die Erstellung der Präsentation reduziert werden musste. Beide Arbeitspakete konnten dennoch fristgerecht abgeschlossen werden.

Insgesamt erwies sich die initiale Kapazitätsplanung als überwiegend realistisch. Die verfügbaren Ressourcen wurden zutreffend eingeschätzt, sodass eine kontinuierliche Umsetzung der Entwicklungsphasen ohne wesentliche Verzögerungen möglich war.

## 8.2 Herausforderungen

Im Verlauf der Entwicklung traten technische und organisatorische Herausforderungen auf, die im Projektkontext berücksichtigt und gelöst werden mussten.

Eine technische Herausforderung bestand in der **plattformübergreifenden Entwicklung** mit React Native. Trotz der gemeinsamen Codebasis mussten plattformspezifische Unterschiede berücksichtigt werden, insbesondere im Bereich der Datenpersistenz zwischen nativen Plattformen und der Web-Umgebung.

Die **Modellierung** der Datenstruktur stellte ebenfalls eine Herausforderung dar. Die Entitäten Zutaten, Gerichte und Mahlzeitenpläne stehen in komplexen Beziehungen zueinander, wodurch eine saubere und erweiterbare Struktur notwendig war. Insbesondere die automatische Berechnung von Einkaufslisten sowie die Aggregation von Nährwerten erforderten eine konsistente Datenbasis.

## 8.3 Lessons Learned

Im Rahmen der Projektarbeit konnten mehrere zentrale Erkenntnisse gewonnen werden, die sowohl für zukünftige Projekte als auch für den praktischen Einsatz relevant sind.

Ein wesentlicher Erfolgsfaktor war die Erstellung eines klar strukturierten **Projektplans** und Pflege in **GitHub**. Die Aufteilung in überschaubare Arbeitspakete ermöglichte eine transparente Fortschrittskontrolle sowie das frühzeitige Erkennen potenzieller Verzögerungen.

Das iterative Vorgehen im Sinne des Rapid Application Development erwies sich als besonders geeignet. Durch die frühe Implementierung funktionsfähiger Prototypen konnten Anfor-

derungen schnell validiert und Anpassungen ohne großen Mehraufwand vorgenommen werden.

Darüber hinaus zeigte sich die Bedeutung einer klaren Architektur und Modularisierung. Die Trennung von UI, Fachlogik und Datenhaltung erleichtert sowohl die Wartbarkeit als auch die Erweiterbarkeit der Anwendung.

Eine zentrale Erkenntnis betrifft die **Integration der Dokumentation** in den Entwicklungsprozess. Die nachgelagerte Erstellung der Dokumentation führte zu einem erhöhten Zeitdruck gegen Ende des Projekts. Für zukünftige Projekte empfiehlt sich daher, die Dokumentation parallel zur Implementierung der einzelnen Prototypen zu erstellen, um den Arbeitsaufwand gleichmäßiger zu verteilen.

Eine entscheidende Erkenntnis betrifft die Arbeitsorganisation und Ressourcenverteilung im Team. Zu Beginn des Projekts lag der Fokus stark auf gemeinsamen **Pair-Programming-Sessions**. Dieser Ansatz war fachlich essenziell, um sich gemeinsam in JavaScript einzuarbeiten, Features detailliert zu diskutieren und ein fundiertes, gemeinsames Verständnis für die Codebasis aufzubauen. Durch den erhöhten Zeitdruck bei der nachgelagerten Erstellung der Projektdokumentation gegen Ende der Frist wurde jedoch die Notwendigkeit und Effizienz einer strategischen **Parallelisierung** deutlich. Für zukünftige Projekte empfiehlt es sich, nach einer initialen, gemeinsamen Findungsphase die Arbeitspakete, insbesondere die Implementierung und die Erstellung der Dokumentation, stärker und deutlich früher parallel aufzuteilen, um Engpässe zu vermeiden.

Ein weiterer Aspekt bleibt die Relevanz einer kontinuierlichen Teamkommunikation. Regelmäßige Abstimmungen und ein transparenter Austausch über den Projektfortschritt tragen maßgeblich dazu bei, Probleme frühzeitig zu identifizieren und effizient zu lösen.

## 9 Anhang

### 9.1 Tabellen

| Verzeichnis        | Zweck / Inhalt                                                | Architektur-Schicht |
|--------------------|---------------------------------------------------------------|---------------------|
| <b>screens/</b>    | Vollbild-Ansichten (Seiten), die den Datenfluss koordinieren. | UI-Schicht          |
| <b>components/</b> | Wiederverwendbare, atomare UI-Bausteine.                      | UI-Schicht          |
| <b>navigation/</b> | Konfiguration des Routings und der Navigationsgraphen.        | UI-Schicht          |
| <b>theme/</b>      | Zentrale Definition von Farben, Abständen und Design-Tokens.  | UI-Schicht          |
| <b>utils/</b>      | Kapselung der Fachlogik, Datenbankzugriffe und Berechnungen.  | Fachlogik-Schicht   |
| <b>data/</b>       | Statische Referenzdaten, Ausgangswerte und Datenbank-Seeds.   | Datenbank-Schicht   |

Tabelle 9: Verzeichnisstruktur und Schichtenzuordnung

| Schicht                           | Technologie                      | Version   | Zweck                                              |
|-----------------------------------|----------------------------------|-----------|----------------------------------------------------|
| <b>Sprache</b>                    | JavaScript (ES6+)                | -         | Kein TypeScript                                    |
| <b>Framework</b>                  | React Native                     | 0.81.5    | UI-Framework für native Plattformen                |
| <b>JavaScript-Bibliothek</b>      | React                            | 19.1.0    | Komponentenmodell, Hooks, State                    |
| <b>Build- &amp; Dev-Tooling</b>   | Expo                             | ~54.0.33  | Managed Workflow, Dev-Server, Builds via Metro     |
| <b>Web-Kompatibilität</b>         | react-native-web                 | 0.21.0    | React-Native-Komponenten im Browser                |
| <b>Navigation</b>                 | React Navigation (Native Stack)  | 7.x       | Stack-basiertes Screen-Routing                     |
| <b>Persistenz (nativ)</b>         | expo-sqlite                      | ~16.0.10  | SQLite-Datenbank auf iOS/Android                   |
| <b>Persistenz (Web)</b>           | AsyncStorage                     | ^2.2.0    | Key-Value-Storage als Fallback im Browser          |
| <b>Grafik / Icons</b>             | react-native-svg                 | 15.12.1   | SVG-basierte Icons und Diagramme                   |
| <b>UI-Erweiterungen</b>           | Kalender, DateTimePicker, Slider | –         | Community-Komponenten aus dem RN-Ökosystem         |
| <b>Gesten</b>                     | react-native-gesture-handler     | ~2.28.0   | Touch-Gesten (Swipe, Drag)                         |
| <b>Hilfsbibliothek</b>            | lodash                           | ^4.17.21  | Allgemeine Utility-Funktionen                      |
| <b>Internationalisierung</b>      | i18next                          | ^25.8.18  | Übersetzungs-Framework (Locales, Pluralisierung)   |
| <b>Internationalisierung</b>      | react-i18next                    | ^16.5.8   | React-Bindings für i18next (Hooks: useTranslation) |
| <b>Internationalisierung</b>      | expo-localization                | ~17.0.8   | Gerätsprache & Region auslesen (Expo-Plugin)       |
| <b>Navigation (Dep.)</b>          | react-native-screens             | ~4.16.0   | Native Screen-Container für React Navigation       |
| <b>Navigation (Dep.)</b>          | react-native-safe-area-context   | ^5.6.2    | Safe-Area-Handling (Notch, Statusbar)              |
| <b>Web-Support</b>                | react-dom                        | 19.1.0    | React-DOM-Renderer für Web-Plattform               |
| <b>Status Bar</b>                 | expo-status-bar                  | ~3.0.9    | Statusleiste konfigurieren (Expo)                  |
| <b>UI-Erweiterungen (Version)</b> | react-native-calendars           | ^1.1313.0 | Kalender-Komponente (bisher ohne Version)          |

|                                   |                                        |                   |                                                    |
|-----------------------------------|----------------------------------------|-------------------|----------------------------------------------------|
| <b>UI-Erweiterungen (Version)</b> | @react-native-community/datetimepicker | 8.4.4             | Datum-/Zeit-Picker (bisher ohne Version)           |
| <b>UI-Erweiterungen (Version)</b> | @react-native-community/slider         | 5.0.1             | Slider-Komponente (bisher ohne Version)            |
| <b>Testing</b>                    | @testing-library/react-native          | ^13.3.3           | Unit-Tests für React-Native-Komponenten            |
| <b>Testing</b>                    | @testing-library/jest-native           | ^5.4.3            | Jest-Matcher für React Native                      |
| <b>Testing</b>                    | jest / jest-expo                       | ^29.7.0 / ~54.0.0 | Test-Runner                                        |
| <b>Linting</b>                    | ESLint + Plugins                       | ^8.57.1           | Statische Code-Analyse (inkl. React, Hooks, JSDoc) |

Tabelle 10: Tech Canvas

| Komponente                     | Aufgabe                                                                                                                                                                             |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>AddDishModal.js</b>         | Modal zum Erstellen/Bearbeiten von Gerichten (Dishes), inkl. Namensgebung, Portionsanzahl und Hinzufügen von Zutaten aus der Datenbank mit automatischer Nährstoffberechnung.       |
| <b>AddIngredientModal.js</b>   | Modal zum Erstellen/Bearbeiten von Zutaten, inkl. Nährwerten pro 100g, Preis, Lagerbestand, Packungsgröße und Zuweisung zu einer Einkaufssektion.                                   |
| <b>BottomNav.js</b>            | Hauptnavigationsleiste am unteren Bildschirmrand mit 5 Tabs (Planner, Cart, Home, Dishes, Ingredients). Unterstützt per Drag & Drop anpassbare Tab-Reihenfolge.                     |
| <b>Button.js</b>               | Wiederverwendbarer Button mit drei Varianten (primary, secondary, outline).                                                                                                         |
| <b>CalorieRing.js</b>          | SVG-basierter Fortschrittsring zur Visualisierung des täglichen Kalorienziels mit numerischer Anzeige im Inneren.                                                                   |
| <b>CompactCalorieMacros.js</b> | Zusammenfassung von Kalorien (kleiner Ring) und Makronährstoffen (Protein, Carbs, Fat) für Übersichtsbereiche.                                                                      |
| <b>ConfirmModal.js</b>         | Generisches Bestätigungsdialogfenster (z. B. für Löschaktionen) mit konfigurierbaren Titel-, Nachricht- und Button-Texten.                                                          |
| <b>InputField.js</b>           | Wiederverwendbares Texteingabefeld mit Label, Platzhalter, Fehleranzeige und Suffix (z. B. „g“, „kcal“),                                                                            |
| <b>MacroCard.js</b>            | Karte zur Anzeige eines einzelnen Makronährstoffs (Name, Wert, Einheit) mit einem horizontalen Fortschrittsbalken.                                                                  |
| <b>MealCard.js</b>             | Karte für einen einzelnen Mahlzeiteintrag mit Mahlzeitentyp, Lebensmittelname, Kalorienanzahl und einem NutrientBar. Diese zeigt einen „+“-Platzhalter wenn kein Eintrag vorhanden. |
| <b>NutrientBar.js</b>          | Horizontaler gestapelter Balken, der die Verteilung von Proteinen, Kohlenhydraten und Fetten als Segmente visualisiert.                                                             |
| <b>PhaseIndicator.js</b>       | Anzeige und Auswahl der Diätphase des Nutzers (Bulk/Cut/Maintenance) mit Datumsauswahl und phasenspezifischer Farbkodierung.                                                        |
| <b>PresetMenuModal.js</b>      | Modal zur Verwaltung von Wochenplan-Vorlagen. Dieses ist für das Speichern der aktuellen Woche, Laden/Erstellen von Presets und Anwenden auf zukünftige Wochen zuständig.           |
| <b>WeightHistory.js</b>        | SVG-Liniendiagramm zur Darstellung des Gewichtsverlaufs in Monats- oder Jahresansicht, inkl. Laden vergangener Einträge und Öffnen des WeightInputModals.                           |
| <b>WeightInputModal.js</b>     | Modal zum Eingeben eines neuen Gewichtseintrags (in kg) mit Auswahl zwischen „Heute“ und einem benutzerdefinierten Datum im Format DD.MM.YYYY.                                      |

Tabelle 11: Übersicht der Komponenten

| <b>Tabelle</b>           | <b>Inhalt / Fokus</b>                                                  | <b>Besonderheiten</b>                                  |
|--------------------------|------------------------------------------------------------------------|--------------------------------------------------------|
| <b>user_profile</b>      | Stammdaten (Name, Geburtstag, Geschlecht, Größe, Gewicht)              | Einzel-Datensatz; appweite Einstellungen als JSON-Blob |
| <b>user_activities</b>   | PAL-Stunden pro Aktivitätsstufe (Schlaf, Sitzen, Gehen etc.)           | FK auf user_profile                                    |
| <b>weight_history</b>    | Tägliche Gewichtseinträge mit Zeitstempel                              | UNIQUE auf date                                        |
| <b>daily_data</b>        | Tagesaggregat-Snapshots als JSON-Blob                                  | UNIQUE auf date                                        |
| <b>ingredients</b>       | Zutaten mit Nährwerten (pro 100 g), Packungsgröße, Lagerbestand, Preis | -                                                      |
| <b>dishes</b>            | Gerichte mit aggregierten Nährwerten und Gesamtpreis                   | Werte werden beim Speichern berechnet                  |
| <b>dish_ingredients</b>  | N:M-Relation Gerichte:Zutaten mit Mengenangabe (Gramm)                 | FK auf dishes und ingredients mit ON DELETE CASCADE    |
| <b>meal_plans</b>        | Mahlzeiteinträge pro Datum und Slot                                    | FK auf dishes                                          |
| <b>week_presets</b>      | Gespeicherte Wochenpläne (Vorlagen)                                    | Primärschlüssel ist eine Text-UUID                     |
| <b>week_pre-set_days</b> | Einzelne Tagesslots innerhalb einer Wochenvorlage                      | FK auf week_presets mit ON DELETE CASCADE              |
| <b>dynamic_slots</b>     | Benutzerdefinierte Mahlzeiten-Slots pro Datum als JSON-Blob            | -                                                      |
| <b>app_settings</b>      | Globaler Key-Value-Speicher für appweite Einstellungen                 | -                                                      |

Tabelle 12: Tabellenübersicht der Datenbank



| Schritt                                       | Soll Start | Ist Start  | Soll Ende  | Ist Ende   |
|-----------------------------------------------|------------|------------|------------|------------|
| <b>Mock-Up and Idea</b>                       | 08.12.2025 | 08.12.2025 | 13.12.2025 | 13.12.2025 |
| <b>Structure and Base Design</b>              | 14.12.2025 | 14.12.2025 | 21.12.2025 | 21.12.2025 |
| <b>Horizontal Prototype</b>                   | 05.01.2026 | 07.01.2025 | 12.01.2025 | 13.01.2025 |
| <b>Database and Profile Prototype</b>         | 13.01.2026 | 14.01.2026 | 19.01.2026 | 19.01.2026 |
| <b>Dashboard Prototype</b>                    | 20.01.2026 | 20.01.2026 | 26.01.2026 | 26.01.2026 |
| <b>Ingredient Prototype</b>                   | 27.01.2026 | 28.01.2026 | 02.02.2026 | 01.02.2026 |
| <b>Dish Prototype</b>                         | 03.02.2026 | 02.02.2026 | 09.02.2026 | 09.02.2026 |
| <b>Meal Plan Prototype</b>                    | 10.02.2026 | 10.02.2026 | 16.02.2026 | 15.02.2026 |
| <b>Shopping Cart and Settings Prototype</b>   | 17.02.2026 | 17.02.2026 | 23.02.2026 | 22.02.2026 |
| <b>Testing</b>                                | 24.02.2026 | 02.03.2026 | 24.02.2026 | 01.03.2026 |
| <b>Deployment</b>                             | 03.03.2026 | 09.03.2026 | 03.03.2026 | 09.03.2026 |
| <b>Projekt- und Architektur Dokumentation</b> | 10.03.2026 | 16.03.2026 | 10.03.2026 | 22.03.2026 |
| <b>Create Presentation</b>                    | 17.03.2026 | 21.03.2026 | 17.03.2026 | 22.03.2026 |

Tabelle 13: Zeitplan (Soll/Ist)

## 9.2 Abbildungen

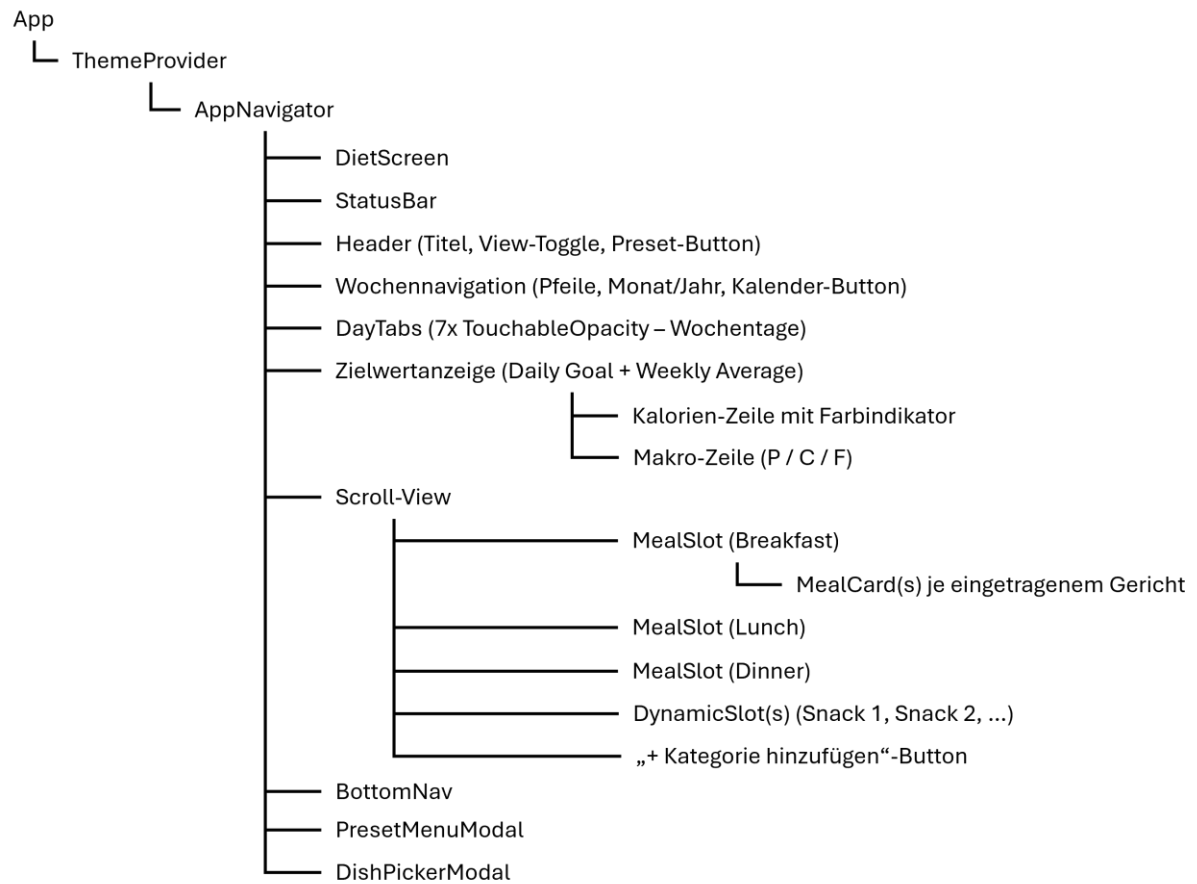


Abbildung 1: Diet-Screen Komponenten-Hierarchie

### 9.3 Quellcode-Ausschnitte

```
const CalorieRing = ({
  current = 0,
  target = 2000,
  size = 200,
  strokeWidth = 12,
}) => {
  const radius = (size - strokeWidth) / 2;
  const circumference = 2 * Math.PI * radius;
  const progress = Math.min(current / target, 1);
  const strokeDashoffset = circumference * (1 - progress);
  const center = size / 2;

  return (
    <View style={[styles.container, { width: size, height: size }]}>
      <Svg width={size} height={size}>
        <G rotation="-90" origin={` ${center}, ${center} `}>
          /* Background circle */
          <Circle
            cx={center} cy={center} r={radius}
            stroke={colors.surface} strokeWidth={strokeWidth}
            fill="transparent"
          />
          /* Progress circle */
          <Circle
            cx={center} cy={center} r={radius}
            stroke={colors.primary} strokeWidth={strokeWidth}
            fill="transparent"
            strokeDasharray={circumference}
            strokeDashoffset={strokeDashoffset}
            strokeLinecap="round"
          />
        </G>
      </Svg>

      /* Center text */
      <View style={styles.centerContent}>
        <Text style={styles.currentValue}>{current.toLocaleString()}</Text>
        <Text style={styles.targetText}>of {target.toLocaleString()} kcal</Text>
      </View>
    </View>
  );
};
```

```

{/* Render all dishes in this slot */}
{mealsArray.length > 0 ? (
  mealsArray.map((meal) => (
    <TouchableOpacity
      key={meal.entryId || `${slot.key}-${meal.dishId}-${Math.random()}`}
      style={[styles.mealCard, { backgroundColor: colors.surface, marginBottom: 8 }]}
      activeOpacity={0.7}
    >
      <FoodIcon color={slot.color} />
      <View style={styles.mealCardContent}>
        <Text style={[styles.mealCardName, { color: colors.text }]}>{meal.dishName}</Text>
        <Text style={[styles.mealCardDetails, { color: colors.textSecondary }]}>
          {Math.round(meal.calories)} kcal • {Math.round(meal.protein)}g Protein
        </Text>
      </View>
      <TouchableOpacity
        style={[styles.deleteButton, { padding: 16 }]}
        onPress={() => handleRemoveMeal(slot.key, meal.entryId)}
        hitSlop={{ top: 10, bottom: 10, left: 10, right: 10 }}
      >
        <CloseIcon color={'#A0A0A0'} />
      </TouchableOpacity>
    </TouchableOpacity>
  ))
) : (
  <Text style={[styles.emptySlotText, { color: colors.textMuted }]}>No meals added</Text>
)}
</View>
);
}}

```

```
const ThemeContext = createContext({
  isDarkMode: true,
  colors: darkTheme,
  toggleTheme: () => { },
  setDarkMode: () => { },
});

export const ThemeProvider = ({ children }) => {
  const [isDarkMode, setIsDarkMode] = useState(true);

  useEffect(() => {
    // Load saved theme preference
    loadThemePreference();
  }, []);

  const loadThemePreference = async () => {
    try {
      const savedTheme = await AsyncStorage.getItem(THEME_STORAGE_KEY);
      if (savedTheme !== null) {
        setIsDarkMode(savedTheme === 'dark');
      }
    } catch (error) {
      console.error('Error loading theme preference:', error);
    }
  };

  const saveThemePreference = async (isDark) => {
    try {
      await AsyncStorage.setItem(THEME_STORAGE_KEY, isDark ? 'dark' : 'light');
    } catch (error) {
      console.error('Error saving theme preference:', error);
    }
  };

  const toggleTheme = () => {
    const newValue = !isDarkMode;
    setIsDarkMode(newValue);
    saveThemePreference(newValue);
  };

  const setDarkMode = (value) => {
    setIsDarkMode(value);
    saveThemePreference(value);
  };

  const colors = isDarkMode ? darkTheme : lightTheme;

  return (
    <ThemeContext.Provider value={{ isDarkMode, colors, toggleTheme, setDarkMode }}>
```

```
        {children}
      </ThemeProvider>
    );
  };

export const useTheme = () => {
  const context = useContext(ThemeProvider);
  if (!context) {
    throw new Error('useTheme must be used within a ThemeProvider');
  }
  return context;
};

export default ThemeContext;
```

---

Quellcode-Ausschnitt 3: Theming

---

```
const Stack = createNativeStackNavigator();

<NavigationContainer>
  <Stack.Navigator>
    initialRouteName={hasProfile ? 'Dashboard' : 'Welcome'}
    screenOptions={{
      headerShown: false,
      contentStyle: { backgroundColor: colors.background },
      animation: 'slide_from_right',
    }}
  >
    <Stack.Screen name="Welcome" component={WelcomeScreen} />
    <Stack.Screen name="ProfileSetup" component={ProfileSetupScreen}
      options={{
        headerShown: true,
        headerTitle: '',
        headerStyle: {
          backgroundColor: colors.background,
        },
        headerTintColor: colors.text,
        headerShadowVisible: false,
      }}
    />
    <Stack.Screen name="Dashboard" component={DashboardScreen} />
    <Stack.Screen name="Diet" component={DietScreen} />
    <Stack.Screen name="Cart" component={CartScreen} />
    <Stack.Screen name="Dishes" component={DishesScreen} />
    <Stack.Screen name="Ingredients" component={IngredientsScreen} />
    <Stack.Screen name="Settings" component={SettingsScreen} />
  </Stack.Navigator>
</NavigationContainer>
```

```
const AppNavigator = () => {
  const [isLoading, setIsLoading] = useState(true);
  const [hasProfile, setHasProfile] = useState(false);

  useEffect(() => {
    const checkProfile = async () => {
      try {
        const profile = await loadUserProfile();
        setHasProfile(!!profile && !!profile.name);
      } catch (error) {
        console.error('Error checking profile:', error);
      } finally {
        setIsLoading(false);
      }
    };

    checkProfile();
  }, []);

  if (isLoading) {
    return (
      <View style={styles.loadingContainer}>
        <ActivityIndicator size="large" color={colors.primary} />
      </View>
    );
  }
}
```

---

Quellcode-Ausschnitt 5: Initialisierung des AppNavigators

---

```
CREATE INDEX IF NOT EXISTS idx_weight_date ON weight_history(date);
CREATE INDEX IF NOT EXISTS idx_daily_date ON daily_data(date);
CREATE INDEX IF NOT EXISTS idx_ingredient_name ON ingredients(name);
CREATE INDEX IF NOT EXISTS idx_dish_name ON dishes(name);
CREATE INDEX IF NOT EXISTS idx_dish_ingredients ON dish_ingredients(dish_id);
CREATE INDEX IF NOT EXISTS idx_meal_plan_date ON meal_plans(date);
CREATE INDEX IF NOT EXISTS idx_preset_days ON week_preset_days(preset_id, day_index);
```

---

Quellcode-Ausschnitt 6: Indizes der Datenbank

---



```
{
  "expo": {
    "name": "fit-fuel-mobile",
    "slug": "fit-fuel-mobile",
    "version": "1.0.0",
    "orientation": "portrait",
    "icon": "./assets/icon.png",
    "userInterfaceStyle": "light",
    "newArchEnabled": true,
    "splash": {
      "image": "./assets/splash-icon.png",
      "resizeMode": "contain",
      "backgroundColor": "#ffffff"
    },
    "ios": {
      "supportsTablet": true
    },
    "android": {
      "package": "com.fitfuel.mobile",
      "adaptiveIcon": {
        "foregroundImage": "./assets/adaptive-icon.png",
        "backgroundColor": "#ffffff"
      },
      "edgeToEdgeEnabled": true
    },
    "web": {
      "favicon": "./assets/favicon.png"
    },
    "plugins": [
      "expo-sqlite",
      "expo-localization"
    ]
  }
}
```

```
const { getDefaultConfig } = require('expo/metro-config');

const config = getDefaultConfig(__dirname);

// Fix lodash module resolution issue with react-native-calendars
config.resolver.extraNodeModules = {
  ...config.resolver.extraNodeModules,
};

// Ensure lodash internal modules are resolved correctly
config.resolver.resolveRequest = (context, moduleName, platform) => {
  // Handle internal lodash imports like './_isPrototype'
  if (moduleName.startsWith('./') && context.originModulePath && context.originModulePath.in-
cludes('node_modules/lodash')) {
    // Let the default resolver handle it
    return context.resolveRequest(context, moduleName, platform);
  }
  return context.resolveRequest(context, moduleName, platform);
};

module.exports = config;
```

---

Quellcode-Ausschnitt 8: metro.config.js

---

## 10 KI-Verzeichnis

### 10.1 Konzeption & Architektur-Entscheidungen

| System          | Prompt                                                                                                                                                                                                      | Verwendung                 |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------|
| Gemini 3 Pro    | How to add people to my private Github repository?                                                                                                                                                          | Rein informativ            |
| Gemini 3 Pro    | how to add my repository in vs code?                                                                                                                                                                        | Rein informativ            |
| Gemini 3 Pro    | <ul style="list-style-type: none"> <li>· Fokus: CrossPlattform</li> <li>· React Native</li> <li>· Hive</li> <li>· fl_chart</li> <li>· Supabase</li> </ul>                                                   | Rein informativ            |
|                 | Kann ich mit diesem Tech-Stack eine mobile Anwendung "Ernährungsplaner für ambitionierte Sportler" schreiben?                                                                                               |                            |
| Gemini 3 Pro    | Warum nicht Sqlite statt hive?                                                                                                                                                                              | Rein informativ            |
| Gemini 3 Pro    | Der Metro Bundler, ist der bei Expo vorhanden und was macht er?                                                                                                                                             | Recherche, rein informativ |
| Claude Opus 4.6 | <p>[Bericht als .pdf-Datei]</p> <p>Die Datenbankstruktur hat sich geändert. Bitte schreibe den Bericht entsprechend um. Gib mir auch eine Tabelle, die eine Übersicht über alle SQLite Tabellen bietet.</p> | Passagen überarbeitet      |

## 10.2 Code-Generierung & Implementierung von Fachlogik

| System            | Prompt                                                                                                                                                                                                                                                                                          | Verwendung                    |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------|
| Gemini 3 Pro      | Was ist eine re export schnittstelle?                                                                                                                                                                                                                                                           | Recherche, rein informativ    |
| Gemini 3 Pro      | Definiere Blob in einem Satz                                                                                                                                                                                                                                                                    | Recherche, rein informativ    |
| Gemini 3 Pro      | Was genau ist Write Ahead Logging?<br>...<br>Du sprichst von Änderung im Log, meinst du mit Log die WAL-File?<br>...<br>Und normalerweise werden die Änderungen immer direkt in die Hauptdatenbank-Datei übertragen? Mit WAL erst bei einem geeigneten Zeitpunkt?                               | Recherche, rein informativ    |
| Gemini 3 Pro      |                                                                                                                                                                                                                                                                                                 | Recherche, rein informativ    |
| Claude Opus 4.6   | Bitte erstelle mir einen Plan zur Implementierung eines Custom-Loggers. Dieser soll gängigen Konventionen folgen.                                                                                                                                                                               | Zur Implementierung verwendet |
| Claude Opus 4.6   | Bitte erarbeite mit mir einen Plan zur Einführung von i18n in dieser Codebase. Die Implementierung sollte gängigen Konventionen folgen, orientiere dich an der offiziellen Dokumentation von i18n. Notiere alle Schritte die nötig sind, damit dieser anschließend implementiert werden können. | Zur Implementierung verwendet |
| Claude Opus 4.6   | Ich möchte, dass du mir einen Plan erstellst, um solche Tests möglichst weitreichend im Programm zu implementieren. Das Ziel ist umfassende E2E und Unit Tests zu implementieren, dokumentiere an welchen Orten im Programm diese angebracht wären.                                             | Zur Implementierung verwendet |
| Claude Sonnet 4.6 | Bitte überprüfe ob sämtliche Funktionen in der Datenbank sowohl für SQLite als auch AsyncStorage implementiert sind, implementiere mit dir restlichen Funktionen, falls einige fehlen sollte und dokumentiere mir, was du gemacht hast.                                                         | Übernommen                    |

### 10.3 UI/UX-Entwicklung & Komponenten-Styling

| System           | Prompt                                                                                                                                                                                                                                                                                                           | Verwendung                 |
|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------|
| Claude Haiku 4.5 | Wie genau funktionieren Hooks                                                                                                                                                                                                                                                                                    | Recherche, rein informativ |
| Claude Haiku 4.5 | Wie verwendet man react-native-svg, um Grafiken zu zeichnen? Gibt es hier einen Standard, oder muss man verstehen, wie man Grafiken darüber zeichnet?                                                                                                                                                            | Recherche, rein informativ |
| Claude Opus 4.5  | In the planner, Please implement that the add meal section is also shown after you added one dish.                                                                                                                                                                                                               | Übernommen                 |
| Claude Opus 4.5  | In the planner tab, i want you to do the following:<br><br>There is the add meal section where you can click on to add a meal. This needs to stay even after adding one meal. So that you can choose multiple smaller dishes for breakfast, lunch, dinner and snacks.                                            | Übernommen                 |
| Claude Opus 4.5  | In den Settings der Button "Clear All Data" soll nach einem bestätigten Confirm-Popup alle Daten zurücksetzen und den User zurück zum ersten Screen (Mit "Get Started") bringen. Außerdem soll beim "Get Started" Screen ein Datepicker für den Geburtstag rechts angezeigt werden.                              | Übernommen                 |
| Claude Opus 4.5  | In settings, change the Sliders of Bulk and Cut. It should not go from -50 to 50. Bulk from 0 to 50 and Cut from 0 to -50. Maintenance should be a fixed value (3.5%) and not shown in the settings. Also, add a section where you see how much calories total these percentage numbers are for better planning. | Übernommen                 |
| Claude Opus 4.6  | Can you add the navigation bar also to the settings menu? (It should just show home). When you press on the home button, you go out of the settings menu as well. And when you go from settings to for example planner and back to home, the home screen should show as well and not the settings.               | Übernommen                 |

## 10.4 Debugging & Problembehebung

| System            | Prompt                                                                                                                                                                                                                                                                                                                                                                                                                                          | Verwendung                 |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------|
| Gemini 3 Pro      | SafeAreaView mit edgeToEdgeEnabled: Was macht das?                                                                                                                                                                                                                                                                                                                                                                                              | Recherche, rein informativ |
| Gemini 3 Pro      | Wieso hat Apple das nicht sondern nur Android?                                                                                                                                                                                                                                                                                                                                                                                                  | Recherche, rein informativ |
| Claude Sonnet 4.6 | Wo werden derzeit die Presets der Wochen gespeichert?                                                                                                                                                                                                                                                                                                                                                                                           | Rein informativ            |
| Claude Sonnet 4.6 | Ich habe noch Commits auf dem lokalen Dev, aber möchte vom origin/dev pullen. Das geht gerade nicht, weil er meine Änderungen behält, ich diese aber verwerfen möchte. wie kann ich das fixen?                                                                                                                                                                                                                                                  | Rein informativ            |
| Claude Sonnet 4.5 | Please suggest a solution for the following problem: Currently, the add meal takes a lot of space and is always there. We thought of adding a plus after each section (Breakfast, lunch, etc.), but then it must be bigger.                                                                                                                                                                                                                     | Rein informativ            |
| Claude Sonnet 4.5 | Okay, next i want to make the dates on top of the planner scrollable. Recommend the best approach to make it work on mobile as well as web. We thought of adding arrows to the left and right, a scrollbar or are there any events for web which also represent swiping (Pressing left mouse button + moving left or right)? And how do we best keep track of the correct days of the dates? We need the month and year above for sure as well. | Rein informativ            |
| Claude Sonnet 4.5 | [Screenshot of the DOM]<br><br>Now i have a whitescreen in the planner. How do I fix this?                                                                                                                                                                                                                                                                                                                                                      | Rein informativ            |
| Claude Opus 4.5   | Please make the section title and the plus bigger, because it needs to be easily clickable on mobile phones as well. Also if you click on the X to remove one meal, and there are two meals with the same name in for example breakfast, both are removed. Please provide unique id's to each element in the array to only delete that specific one.                                                                                            | Übernommen                 |
| Claude Opus 4.5   | The Login window is now removed. However, the startday of the planner is still not the one selected in the get started screen. Please fix that. You might need to reload the page so that you can test. If you have further questions, feel free to ask.                                                                                                                                                                                        | Übernommen                 |

## 10.5 Code-Qualität, Refactoring & Testing

| System            | Prompt                                                                                                                                                                           | Verwendung                 |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------|
| Claude Sonnet 4.6 | Um einen nahtlosen Übergang zu gewährleisten, wurde eine Migrationsroutine implementiert, die bestehende AsyncStorage-Daten beim ersten Start in die SQLite-Datenbank überführt. | Recherche, rein informativ |
|                   | Wo finden wir diese Routine?                                                                                                                                                     |                            |
| Claude Sonnet 4.6 | eslint:recommended, plugin:react/recommended und plugin:react-hooks/recommended                                                                                                  | Recherche, rein informativ |
|                   | Was machen diese 3                                                                                                                                                               |                            |

## 10.6 Textredaktion & Dokumentationsstruktur

| System            | Prompt                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | Verwendung            |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|
| Gemini 3 Pro      | Die Applikation „Fit Fuel Mobile“ richtet sich primär an die Zielgruppe junge Erwachsene im Alter von 25-40 Jahren<br><br>junge Erwachsene im Alter von 25-40 Jahren mit anführungszeichen oder ohne?                                                                                                                                                                                                                                                                                                                                                  | Rein informativ       |
| Gemini 3 Pro      | Würdest du Dateien in Anführungszeichen oder kursiv darstellen                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | Rein informativ       |
| Claude Sonnet 4.6 | Beschreibe detailliert, was diese App tut. Nicht auf Code-Ebene, sondern auf Logik-Ebene, sodass ich anhand dessen einen Bericht schreiben kann                                                                                                                                                                                                                                                                                                                                                                                                        | Rein informativ       |
| Claude Haiku 4.6  | Wofür steht das ^ und ~ symbol?                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | Rein informativ       |
| Claude Opus 4.6   | Gib mir Stichpunkte für die Umsetzung zu 3 Frontend: Struktur / Bausteine                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | Rein informativ       |
| Gemini 3 Pro      | Der Kernprozess, den die App abbildet, beginnt mit der präzisen Errechnung des individuellen Kalorien- und Makronährstoffbedarfs auf Basis persönlicher Messwerte. Darauf aufbauend generiert die Anwendung passende Rezepte und automatisierte Einkaufszettel.<br><br>[Erste Fassung von Kapitel 1]<br><br>Das ist nicht mehr richtig. Passe den Text entsprechend an.<br><br>Bitte schreibe möglichst wenig redundant. Stattdessen solltest du maximal 2 Seiten schreiben, etwas kürzer ist auch nicht schlimm. Wichtig ist präzise und vollständig. | Passagen überarbeitet |
| Gemini 3 Pro      | [Bericht als .docx-Datei]                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | Passagen überarbeitet |
| Gemini 3 Pro      | Lese diesen bericht Korrektur und fasse ihn etwas zusammen, ohne die wichtigen Punkte zu verlieren<br><br>Die Bildschirmausrichtung ist auf "portrait" festgelegt, da das UI-Layout der Screens nicht für Landscape-Darstellung ausgelegt ist. „edgeToEdgeEnabled“ auf Android gibt der App den gesamten Bildschirmbereich einschließlich der Systemleisten frei, was in Verbindung mit „SafeAreaView“ in jedem Screen für ein konsistentes Layout ohne verdeckte Inhalte sorgt.                                                                       | Passagen überarbeitet |
| Gemini 3 Pro      | ergänze hier, dass Apple das schon im Standard macht<br><br>Schreibe in einem Satz:<br><br>Die Datei metro.config.js erweitert die Expo-Standardkonfiguration um eine benutzerdefinierte resolveRequest-Funktion. Diese wurde eingeführt, um einen Auflösungskonflikt mit internen lodash-Modulen zu beheben, der durch die Abhängigkeit „react-native-calendars“ entsteht. Lodash verwendet intern relative Imports wie './_isPrototype', die der standard Metro-Resolver unter                                                                       | Passagen überarbeitet |



|                 |                                                                                                                                                                                                                                                                                                                                                                                                    |                       |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|
|                 | bestimmten Umständen nicht korrekt auflöst. Die benutzerdefinierte Logik erkennt solche Imports anhand des Ursprungspaths und delegiert sie explizit an den Standard-Resolver zurück:                                                                                                                                                                                                              |                       |
| Gemini 3 Pro    | Schreibe bitte einen Einleitungssatz für den Technik-Stack und für 1 Anforderungen und Ziele                                                                                                                                                                                                                                                                                                       | Passagen überarbeitet |
| Claude Opus 4.6 | [Anforderungen als .pdf-Datei]<br><br>Das sind die Anforderungen, die wir an unsere Dokumentation haben. Gehe die Dateien durch, und schau, welchen Technik Stack wir hier verwenden. Anschließend schreibe meinen Bericht beim Punkt 2 Technic Stack fort. Bitte verwende ähnliche Sprache wie von mir, es sollte professionell sein, aber kein marketing. Möglichst neutral ohne Beschönigungen. | Passagen überarbeitet |
| Claude Opus 4.6 | Erstelle eine Tabelle der einzelnen Komponenten in components sowie deren Aufgabe.                                                                                                                                                                                                                                                                                                                 | Passagen überarbeitet |